

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	N Goutham	Reg. No.:	192472024
Date:	06-08-2026	Duration:	60 minutes

Code of Conduct

I N Goutham/192472024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Goutham

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

1. System Requirement Analysis

1.1 System Overview

The proposed Open-Source Contribution Tracker and Recommendation System is intended to help beginner developers enter the open-source software ecosystem in a structured way. The system addresses the difficulty of selecting suitable repositories and beginner-friendly issues from a very large number of projects. It combines developer skill information with GitHub repository and issue information to provide personalized recommendations.

The architecture must therefore support several related activities: user profile management, GitHub integration, issue discovery, recommendation, contribution tracking, skill progression, notifications, portfolio generation, and community insights. These functions involve both internal application components and an external GitHub API.

1.2 Architecture-Influencing Functional Requirements

Requirement	Architectural Influence
User and Skill Profile	The system must store programming languages, technologies, interests, and skill level so that recommendations can be personalized.

GitHub Integration	The system must retrieve repository, issue, label, pull request, and contribution information through an external API.
Recommendation	The system must match user skills and interests with suitable beginner-friendly issues and rank the results.
Difficulty Scoring	The system must estimate issue or repository complexity to support appropriate recommendations.
Contribution Tracking	The system must consolidate issues, pull requests, merge status, and contribution history.
Dashboard and Progress	The system must present contribution frequency, technology diversity, and progression information.
Notifications	The system must alert users when relevant beginner-friendly opportunities are available.
Portfolio Generation	The system must organize contribution information into a professional portfolio summary.

1.3 Key Architectural Needs

- Separation of user-interface responsibilities from business and recommendation logic.
- Secure control of communication with the GitHub API so that credentials and tokens are not exposed in the client.
- A persistent data store for user profiles, repository data, issues, contributions, and notifications.
- Modularity so recommendation and difficulty-scoring logic can be improved independently.
- An architecture that can support more users, repositories, and contribution records over time.

2. Non-Functional Requirements and Quality Attributes

2.1 Scalability

The system should be able to support growth in the number of users, repositories, issues, and contribution records. A layered design allows application responsibilities to be separated so that high-

load components can be optimized independently. Repository and issue retrieval can also be controlled through caching or scheduled synchronization when required.

2.2 Maintainability

The architecture should keep presentation, application logic, recommendation logic, data access, and external API integration as distinct responsibilities. This reduces the effect of changes in one part of the system on other parts. For example, the recommendation algorithm can be changed without redesigning the user interface.

2.3 Reliability

The system depends on GitHub as an external service, so the architecture must handle temporary API failures, incomplete responses, and communication errors. The application should validate external responses and preserve locally stored contribution information so that a temporary GitHub failure does not unnecessarily destroy previously collected data.

2.4 Availability

The system should remain available for users to view their profiles, stored contribution history, and previously retrieved information even when external data cannot be refreshed temporarily. External API-dependent functions can report a controlled service-unavailable message rather than failing the entire application.

2.5 Performance

Recommendation and search operations should be responsive. Frequently accessed information can be stored locally after retrieval from GitHub, reducing repeated external requests. The architecture also separates the recommendation component from the presentation layer so that recommendation processing does not require the user interface to directly communicate with GitHub.

2.6 Security

The architecture must protect user authentication information and GitHub credentials or access tokens. Client applications should communicate with the backend using secure HTTPS connections. The backend should control access to protected operations and validate incoming requests before accessing stored information or external APIs.

2.7 Summary of Quality Attributes

Quality Attribute	Architectural Response
Scalability	Modular layers and independently optimizable components.

Maintainability	Separation of concerns and clear component responsibilities.
Reliability	Error handling and persistence of previously retrieved data.
Availability	Local access to stored information and controlled external-service failure handling.
Performance	Efficient data access and reduced unnecessary external API calls.
Security	Backend-controlled API access, authentication, authorization, and HTTPS.

3. Selection of Suitable Software Architecture

3.1 Selected Architecture

The selected architecture for the proposed system is a Hybrid Layered Architecture combining a client-server approach with clearly separated application layers and external API integration. The architecture contains a presentation layer, application layer, specialized recommendation and contribution components, a data layer, a database, and an external GitHub API.

3.2 Why Hybrid Layered Architecture is Suitable

The proposed system contains multiple responsibilities that should not be implemented as one tightly coupled component. The user interface is responsible for interaction, while the backend coordinates business operations. Recommendation and difficulty scoring have specialized processing responsibilities, and the database manages persistent information. GitHub is an external system that must be accessed through a controlled integration layer.

A layered organization provides separation of concerns. The presentation layer does not need to know database implementation details, while the database does not need to understand user-interface behavior. This improves maintainability and allows each layer to evolve independently.

3.3 Architectural Layers

Layer/Component	Main Responsibility
Presentation Layer	Web or mobile interface, dashboard, search, profile, recommendation and portfolio views.
Application Layer	Authentication, user profile operations, request

	coordination, contribution operations and API coordination.
Recommendation and Analysis Components	Skill matching, issue ranking, difficulty scoring, skill progression and community indicators.
Data Layer	Data access and persistence operations for users, skills, repositories, issues, contributions and notifications.
External Integration	Controlled communication with GitHub API through secure REST/HTTPS requests.

3.4 Comparison with Alternative Architectures

Architecture	Suitability for this Project
Simple Monolithic	Easy to begin with, but responsibilities can become tightly coupled as recommendation, tracking and integration features grow.
Microservices	Highly scalable, but introduces deployment, communication, monitoring and operational complexity that may be unnecessary for the initial system.
Event-Driven	Useful for notifications and asynchronous processing, but alone does not provide the complete structural separation required by the system.
Hybrid Layered	Provides clear separation, manageable complexity, external API integration, and a practical path for future scaling.

Therefore, the hybrid layered approach provides an appropriate balance between simplicity, modularity, maintainability, and future scalability for the proposed beginner-focused platform.

4. Software Architecture Diagram and Component Design

The architecture diagram below illustrates the major components, data stores, external system, and communication paths of the proposed platform. The beginner developer interacts with the presentation layer. Requests are handled by the application layer, which coordinates specialized modules and access to persistent data. GitHub information is retrieved through a controlled external API integration.

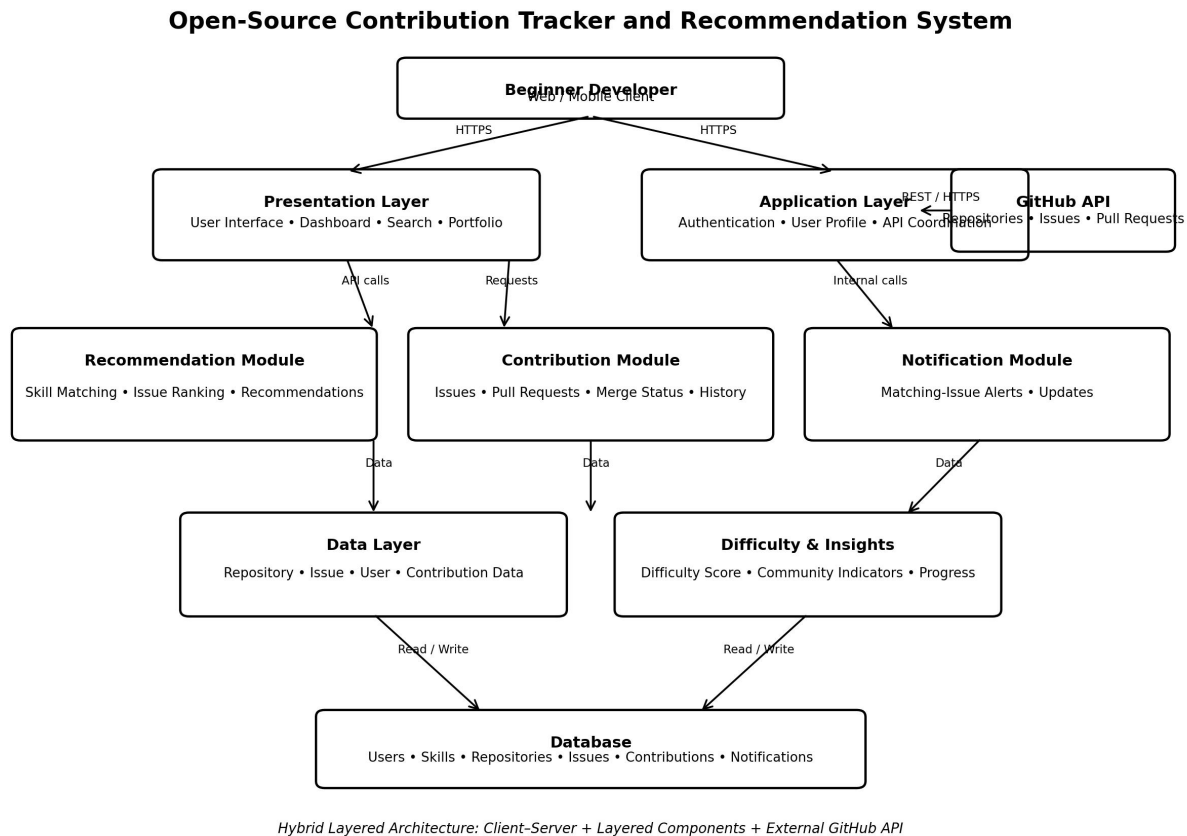


Figure 1: Proposed Hybrid Layered Architecture

4.1 Main Architectural Components

- **Beginner Developer:** Initiates actions such as profile management, issue search, recommendation viewing, contribution tracking, and portfolio generation.
- **Presentation Layer:** Provides the web/mobile interface and displays information from the backend.
- **Application Layer:** Coordinates requests, authentication, business operations, and communication between internal components.
- **Recommendation Module:** Matches developer skills and interests with relevant beginner-friendly issues.
- **Contribution Module:** Maintains issue, pull request, merge-status, and contribution-history operations.

- Notification Module: Handles alerts for relevant new issues and contribution updates.
- Difficulty and Insights Module: Produces issue difficulty indicators, progression measures, and available community insights.
- Data Layer and Database: Stores user profiles, skills, repositories, issues, contributions, and notification information.
- GitHub API: Supplies external repository, issue, pull request, label, and contribution-related information.

5. Component Interactions and System Workflow

5.1 User Profile Workflow

When a beginner developer registers and creates a profile, the presentation layer sends the information to the application layer. The application validates the request and stores the profile and skill information through the data layer. The stored skills and interests later become inputs to the recommendation process.

5.2 GitHub Data Workflow

When GitHub information is required, the application layer communicates with the GitHub API using secure REST/HTTPS communication. Repository, issue, label, pull request, and contribution information received from GitHub is validated and processed before being stored or used by internal components.

5.3 Recommendation Workflow

The recommendation process begins with the user's stored skill profile. The system retrieves relevant issue information and filters beginner-friendly issues. The recommendation module compares issue technologies and available characteristics with the user's skills and interests. The difficulty component provides an estimated complexity indicator. The final ranked results are returned to the presentation layer.

5.4 Contribution Tracking Workflow

After the user connects GitHub, contribution-related information can be retrieved and synchronized. The contribution module organizes issues, pull requests, and merge status. The resulting records are stored so that the dashboard can display a centralized history across multiple repositories.

5.5 Notification Workflow

The system periodically checks or receives updated issue information through the GitHub integration. When a newly available issue matches a user's profile, the notification component prepares an alert. The user receives the notification through the application's notification interface.

5.6 Overall Data Flow

Step	Interaction
1	Developer sends a request through the presentation layer.
2	Application layer authenticates and validates the request.
3	Required data is obtained from the database or GitHub API.
4	Recommendation, tracking, scoring, or notification modules process the information.
5	Results are stored when required.
6	Processed information is returned to the presentation layer.
7	The developer views recommendations, progress, contributions, or notifications.

5.7 Communication Mechanisms

- Client to application layer: HTTPS-based web/mobile requests.
- Application to GitHub: REST API over HTTPS.
- Application to database: controlled data-access operations.
- Internal components: application-level service calls and structured data exchange.

6. Quality Attribute Analysis

6.1 Scalability

The architecture can scale by improving individual layers and modules according to demand. For example, recommendation processing can be optimized separately from profile management. Database indexing and efficient data access can support growth in contribution records. The architecture also allows additional clients or future services to use the same backend.

6.2 Security

Security is addressed by keeping protected operations on the backend. The client should not directly access the database or handle sensitive GitHub credentials unnecessarily. Authentication and authorization should be enforced at the application layer, and communications should use HTTPS.

6.3 Performance

Performance is supported by separating external data retrieval from presentation responsibilities. Previously retrieved repository and issue information can be persisted and reused when appropriate. This reduces unnecessary calls to GitHub and helps recommendation requests remain responsive.

6.4 Reliability

Reliability is improved by validating external API responses, handling communication errors, and preserving important previously synchronized information. A temporary GitHub outage should affect data refresh rather than making the entire application unusable.

6.5 Maintainability

The separation of layers and components makes the system easier to maintain. A developer can modify the recommendation algorithm, notification logic, or data-access implementation without redesigning the presentation layer. Clear interfaces between components reduce coupling.

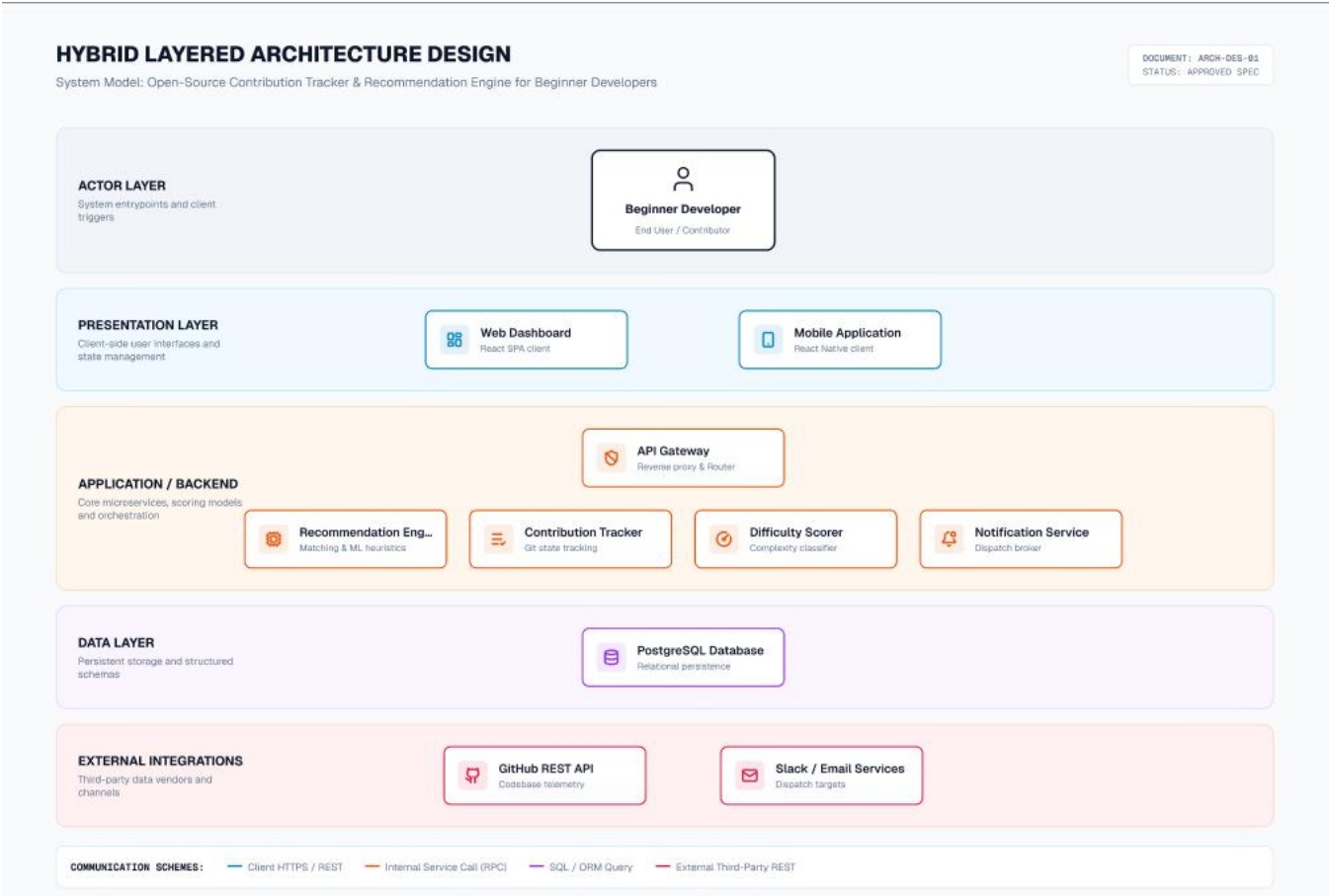
6.6 Availability

The system can maintain useful functionality by serving stored profile, contribution, and recommendation-related data when possible. External API failures can be isolated and reported to users in a controlled manner.

6.7 Quality Attribute Summary

Attribute	Expected Architectural Benefit
Scalability	Components can be optimized and expanded as usage grows.
Security	Sensitive operations remain controlled by the backend.
Performance	Local persistence reduces unnecessary external API calls.
Reliability	External failures are isolated through validation and error handling.
Maintainability	Layered separation reduces coupling.
Availability	Stored information remains accessible during temporary API issues.

FIGMA TOOL ARCHITECTURE:



7. Architectural Justification, Trade-Offs and Future Enhancements

7.1 Architectural Justification

The hybrid layered architecture is appropriate because the proposed system combines a user-facing application, recommendation logic, contribution tracking, persistent data, notifications, and a dependency on the GitHub API. Separating these responsibilities makes the system easier to understand, test, maintain, and extend.

The architecture also supports the project's long-term objective of expanding from beginner recommendations to intermediate and advanced contributor recommendations. New recommendation strategies can be introduced without changing the fundamental user-interface and data-management responsibilities.

7.2 Trade-Offs

A major advantage of the selected approach is that it provides architectural structure without the operational complexity of a complete microservices architecture. A microservices solution could provide strong independent scalability, but it would also require additional deployment, service discovery, monitoring, communication, and failure-management mechanisms. For an initial academic system, that additional complexity may not provide proportional benefit.

A simple monolithic design would be easier to start with, but could make the recommendation, notification, contribution, and GitHub integration responsibilities more tightly coupled. The hybrid layered approach provides a middle ground: it retains manageable deployment complexity while maintaining clear component boundaries.

7.3 Future Enhancements

- Extend recommendations from beginner-level issues to intermediate and advanced issues.
- Introduce more advanced machine-learning techniques for personalized issue ranking.
- Add asynchronous event processing for large-scale notifications and synchronization.
- Introduce caching for frequently requested repository and issue information.
- Support additional code-hosting platforms in addition to GitHub.
- Provide richer community analytics using project activity and response indicators.
- Add additional portfolio formats and integrations for professional profiles.

7.4 Conclusion

The proposed architecture provides a structured foundation for the Open-Source Contribution Tracker and Recommendation System for Beginner Developers. The hybrid layered approach separates

presentation, application, recommendation, contribution, data, and external integration responsibilities. This separation supports the required quality attributes of scalability, security, performance, reliability, maintainability, and availability.

The architecture also provides a practical path for future enhancement. As the system grows, selected modules can be optimized or separated further without redesigning the complete application. Therefore, the selected architecture is suitable for the functional and non-functional requirements of the proposed system and provides a maintainable foundation for future development.

8. Architecture Design Tool

The software architecture diagram can be recreated and edited using a diagramming tool such as diagrams.net (draw.io). The final editable diagram should be inserted into the assessment report together with the component and interaction explanations. The diagram should clearly show the client, application layers, specialized modules, database, GitHub API, and communication paths.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components,	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient	Diagram is incomplete, unclear, or technically incorrect.

	interfaces, and interactions accurately represented.		detail.	
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25